```
  background: rgba(255,255,255,0);
  border-width: 1px;
  border-style: solid;
  border-color: rgba(0,0,0,0);
}
```

8.1.2 HTML

The HTML is pretty simple: a single `<div>` with the ID value of `"traditional"` in your `<body>` element, as shown in the following listing. You still need a reference to d3.js, but otherwise it's a Spartan HTML page. You'll either modify or add new elements to that div for every example.

Listing 8.2 chapter8.html

```
<!doctype html>
<html>
<script src="d3.v3.min.js" type="text/JavaScript"></script>
<body>
<div id="traditional">
</div>
</body>
</html>
```

8.2 Spreadsheet

Let's assume we want to take the tweets data that we've been working with throughout the book and present it as a spreadsheet. It may help to first think of spreadsheets as a kind of information visualization. They have an x-axis (columns) and a y-axis (rows) and visual channels to express information (not only color applied to text and cells but also position and font styling). This is especially true of large spreadsheets, because they also use aggregated functions to tally results.

8.2.1 Making a spreadsheet with table

The easiest way to make a spreadsheet is to use the HTML `<table>` element and data-binding to create rows and cells. As we've done previously, we create key values by using `d3.keys` on one of the entries in our dataset, which will be the venerable tweets.json. After we bind the dataset to the table, we need to create individual cells. We can accomplish this by taking each JSON object and applying `d3.entries()` to it, which turns an object into an array of key-value pairs perfectly suited for D3 data-binding.

Listing 8.3 Spreadsheet example

```
d3.json("tweets.json",function(error,data) {
                  createSpreadsheet(data.tweets)});

      function createSpreadsheet(incData) {

        var keyValues = d3.keys(incData[0]);
```

This won't work if your objects have differing numbers of attributes, but usually that's not the case.